Markdown Studio — Feature Demo

Open this file and run **Markdown Studio: Open Secure Preview** (`Cmd+Shift+P`).

---

1. Markdown Rendering

Bold, *italic*, ~~strikethrough~~, `inline code`

> Blockquotes work too.

Feature	Status
Markdown	✅
Mermaid	✅
PlantUML	✅
SVG	✅
PDF Export	✅

1. Ordered list item
2. Another item
3. Third item

- Unordered item
- Another item

- Item

- 日本語

---

2. Mermaid Diagrams

Markdown Studio Architecture

```
flowchart TD
    A[Markdown Source] --> B[markdown-it Parser]
    B --> C{Fenced Block?}
    C -- mermaid --> D[Mermaid Placeholder]
    C -- plantuml/puml --> E[PlantUML JAR → SVG]
    C -- svg --> F[SVG Sanitizer]
    C -- code --> G[highlight.js]
    C -- none --> H[HTML Output]
    D --> I[sanitize-html]
    E --> I
    F --> I
    G --> I
```

```
    H --> I
    I --> J[buildHtml + CSP Nonce]
    J --> K[Webview Preview]
    J --> L[Playwright → PDF Export]
```

Extension Activation Flow

```
sequenceDiagram
    participant VS as VS Code
    participant Ext as Extension Host
    participant DM as DependencyManager
    participant WV as Webview

    VS->>Ext: activate()
    Ext->>DM: ensureAll()
    DM-->>Ext: Java ✅  Chromium ✅
    Ext->>VS: Register commands
    VS->>Ext: openPreview
    Ext->>Ext: renderMarkdownDocument()
    Ext->>WV: HTML + CSP + Nonce
    WV->>WV: Mermaid client-side render
```

---

3. PlantUML Diagrams

Rendered locally via bundled JAR. No remote server.

Extension Component Diagram

```
@startuml
skinparam componentStyle rectangle
skinparam defaultFontSize 14

package "Markdown Studio" {
  [Extension Host] as ext
  [Markdown Parser] as parser
  [Mermaid Renderer] as mermaid
  [PlantUML Renderer] as plantuml
  [SVG Sanitizer] as svg
  [HTML Builder] as html
  [PDF Exporter] as pdf
  [Webview Preview] as preview
}

package "External Dependencies" {
  [Amazon Corretto JDK] as java
  [Playwright Chromium] as chromium
  [PlantUML JAR] as jar
}
```

```
ext --> parser
parser --> mermaid
parser --> plantuml
parser --> svg
parser --> html
html --> preview
html --> pdf

plantuml --> jar
jar --> java
pdf --> chromium
@enduml
```

Document Processing Sequence

```
@startuml
skinparam defaultFontSize 14

actor Developer
participant "VS Code" as vsc
participant "Extension" as ext
participant "PlantUML JAR" as jar
participant "Webview" as wv

Developer -> vsc : Open .md file
Developer -> vsc : Cmd+Shift+P → Preview
vsc -> ext : openPreview command
ext -> ext : scanFencedBlocks()
ext -> ext : renderMermaidBlock() → placeholder
ext -> jar : java -jar plantuml.jar -tsvg
jar --> ext : SVG output
ext -> ext : sanitizeSvg()
ext -> ext : sanitizeHtmlOutput()
ext -> ext : buildHtml() + CSP nonce
ext -> wv : Set webview HTML
wv -> wv : Mermaid client-side render
wv --> Developer : Live preview
@enduml
```

---

4. Inline SVG (Sanitized)

Dangerous elements (`<script>`, `<foreignObject>`, event handlers) are stripped automatically.

```
<svg viewBox="0 0 360 80" xmlns="http://www.w3.org/2000/svg" width="100%">
  <rect x="5" y="5" width="110" height="70" rx="10" fill="#4CAF50" />
  <text x="60" y="48" text-anchor="middle" fill="white" font-size="18"
font-weight="bold">Parse</text>
```

```svg
  <rect x="125" y="5" width="110" height="70" rx="10" fill="#2196F3" />
  <text x="180" y="48" text-anchor="middle" fill="white" font-size="18"
font-weight="bold">Render</text>
  <rect x="245" y="5" width="110" height="70" rx="10" fill="#FF9800" />
  <text x="300" y="48" text-anchor="middle" fill="white" font-size="18"
font-weight="bold">Export</text>
</svg>
```

---

5. Syntax Highlighting

```typescript
import * as vscode from 'vscode';

export function activate(context: vscode.ExtensionContext): void {
  const depManager = new DependencyManager();
  const status = await depManager.ensureAll(context);

  context.subscriptions.push(
    vscode.commands.registerCommand('markdownStudio.openPreview', async ()
=> {
      await openPreviewCommand(context);
    }),
    vscode.commands.registerCommand('markdownStudio.exportPdf', async ()
=> {
      await exportPdfCommand(context);
    })
  );
}
```

```json
{
  "markdownStudio.plantuml.mode": "bundled-jar",
  "markdownStudio.java.path": "java",
  "markdownStudio.export.pageFormat": "A4",
  "markdownStudio.security.blockExternalLinks": true
}
```

```python
# PlantUML rendering is also useful for Python projects
from dataclasses import dataclass

@dataclass
class DiagramConfig:
    mode: str = "bundled-jar"
    java_path: str = "java"
    timeout_ms: int = 15000
```

---

6. Security Model

- ✅ No external API calls
- ✅ No SaaS dependency or CDN assets
- ✅ Restrictive CSP with random nonce
- ✅ HTML sanitization before rendering
- ✅ SVG sanitization strips scripts, event handlers, foreign objects
- ✅ External links and images blocked by default

External image (blocked by policy)

Remote image

The image above is replaced with a policy notice in the preview.

External link (blocked by policy)

External link — blocked by default ✋

---

7. Theme Adaptability

Switch between **light** and **dark** mode in VS Code (`Cmd+K Cmd+T`) to see how the preview adapts:

- Mermaid diagrams automatically switch between light and dark themes
- SVG elements use colors chosen for visibility in both themes
- Code blocks use theme-aware syntax highlighting
- PlantUML output receives CSS overrides for dark mode

---

8. Diagram Type Catalog

All diagram types below are verified to render correctly with the bundled PlantUML + Smetana engine (no Graphviz required).

▶ Class Diagram

```
@startuml
class Animal {
  +name: String
  +speak(): void
}
class Dog extends Animal {
  +fetch(): void
}
class Cat extends Animal {
  +purr(): void
}
@enduml
```

▶ Activity Diagram

```
@startuml
start
:Parse Markdown;
if (Has diagrams?) then (yes)
  fork
    :Render Mermaid;
  fork again
    :Render PlantUML;
  end fork
else (no)
  :Skip diagram rendering;
endif
:Sanitize HTML;
:Build preview;
stop
@enduml
```

▶ State Diagram

```
@startuml
[*] --> Idle
Idle --> Parsing : openPreview
Parsing --> Rendering : fencedBlockFound
Rendering --> Composing : allBlocksRendered
Composing --> Displaying : htmlBuilt
Displaying --> Idle : panelClosed
Displaying --> Parsing : documentChanged
@enduml
```

▶ Use Case Diagram

```
@startuml
actor Developer
Developer --> (Write Markdown)
Developer --> (Preview Document)
Developer --> (Export PDF)
Developer --> (Validate Environment)
@enduml
```

▶ Timing Diagram

```
@startuml
robust "Extension Host" as EH
robust "Webview" as WV
```

```
@0
EH is Idle
WV is Empty

@100
EH is Parsing

@200
EH is Rendering
WV is Loading

@300
EH is Idle
WV is Displaying
@enduml
```

▶ Mind Map

```
@startmindmap
* Markdown Studio
** Rendering
*** Mermaid
*** PlantUML
*** SVG Sanitizer
*** highlight.js
** Export
*** PDF (Playwright)
** Security
*** CSP Nonce
*** HTML Sanitization
*** External Link Blocking
@endmindmap
```

▶ Gantt Chart

```
@startgantt
[Parse Markdown] lasts 1 day
[Render Diagrams] starts at [Parse Markdown]'s end and lasts 2 days
[Sanitize HTML] starts at [Render Diagrams]'s end and lasts 1 day
[Build Preview] starts at [Sanitize HTML]'s end and lasts 1 day
@endgantt
```

▶ Object Diagram

```
@startuml
object ExtensionConfig {
```

```
    plantUmlMode = "bundled-jar"
    javaPath = "java"
    pageFormat = "A4"
    blockExternalLinks = true
  }
  object DependencyStatus {
    allReady = true
    javaPath = "/path/to/java"
    browserPath = "/path/to/chromium"
  }
  ExtensionConfig -- DependencyStatus
  @enduml
```

▶ Mermaid: Pie Chart

```
pie title Markdown Studio Components
    "Rendering" : 40
    "Security" : 25
    "Export" : 20
    "Infrastructure" : 15
```

▶ Mermaid: Git Graph

```
gitGraph
    commit id: "init"
    commit id: "add-preview"
    branch feature/plantuml
    commit id: "plantuml-renderer"
    commit id: "smetana-engine"
    checkout main
    merge feature/plantuml
    commit id: "pdf-export"
```

---

9. PDF Export

This entire document can be exported to PDF:

1. Open this file in VS Code
2. `Cmd+Shift+P` → **Markdown Studio: Export PDF**
3. A `demo.pdf` will be generated next to this file

The PDF uses the same HTML pipeline as the preview.

Markdown Studio v0.1.0